Computer System Engineering (CSL301)
LAB-3
2025-26 M

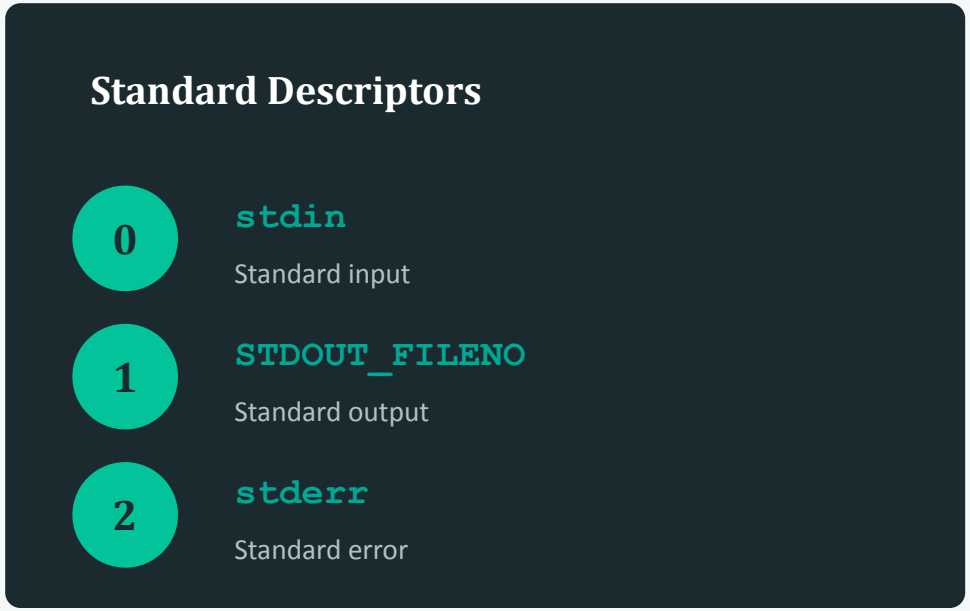
We'll Start the discussion with

1. **File Descriptors**
   The integer handles Linux uses for every open file
2. **open()**
   Turning a path into a usable descriptor
3. **perror() & errno**
   Reporting exactly what went wrong
4. **read()**
   Pulling bytes from a descriptor into memory
5. **write()**
   Pushing bytes from memory to a descriptor
6. **close()**
   Releasing the descriptor back to the OS

File Descriptors

Linux represents every open file as a small non-negative integer.

- A file descriptor (fd) is just an index the kernel uses to track an open file, socket, or pipe for your process.
- open() hands you a new fd. Every later call — read(), write(), close() — refers back to that same integer.




```
open()  →  returns the NEXT available integer (often 3, 4, 5 …)
```

open()

Turns a file path into a file descriptor.

```
int fd = open(path, flags, mode);
// returns -1 on failure
```

path — the file to open

flags — how to open it (see table)

mode — permissions, only used with O_CREAT (e.g. 0644)

Common Flags

`O_RDONLY`	Open for reading only
`O_WRONLY`	Open for writing only
`O_RDWR`	Open for reading and writing
`O_CREAT`	Create the file if it doesn't exist
`O_TRUNC`	Truncate existing contents to 0
`O_APPEND`	Writes go to the end of the file

Error Handling: perror() & errno

Almost every system call can fail — check it, every time.

```
fd = open(argv[1], O_RDONLY);
if (fd == -1) {
    perror("open");
    return 1;
}
```

A failed system call sets the global variable **`errno`** to a code identifying the problem.

`perror("open")` prints your label plus a human-readable reason, e.g.:

```
open: No such file or directory
```

Why bother checking?

- open() returns -1 if the file is missing, unreadable, or permission is denied.
- read() and write() also return -1 on error — the same pattern applies.
- Skipping the check means your program silently uses an invalid fd (-1) and crashes or corrupts data later.
- This one pattern — check, then perror() — is used consistently across all syscalls.

read()

Pulls bytes from a file descriptor into a buffer.

```
ssize_t n = read(fd, buffer, count);
// n = bytes actually read | 0 = end of file | -1 = error
```

1

Return value

n may be LESS than count — the OS is free to hand back fewer bytes than you asked for.

2

End of file

read() returns 0 once there is nothing left to read — that's your loop's exit signal.

3

No auto null-terminator

The buffer is raw bytes, not a C string. Use n to know how much is valid data.

write()

Pushes bytes from a buffer out to a file descriptor.

```
ssize_t n = write(fd, buffer, count);
// n = bytes actually written | -1 = error
```

Same fd concept, opposite direction — you can write to a file, or to **`STDOUT_FILENO`** (fd 1) to print straight to the terminal without printf().

Like read(), a successful write() can report fewer bytes written than requested for large writes — production code loops until count is satisfied.

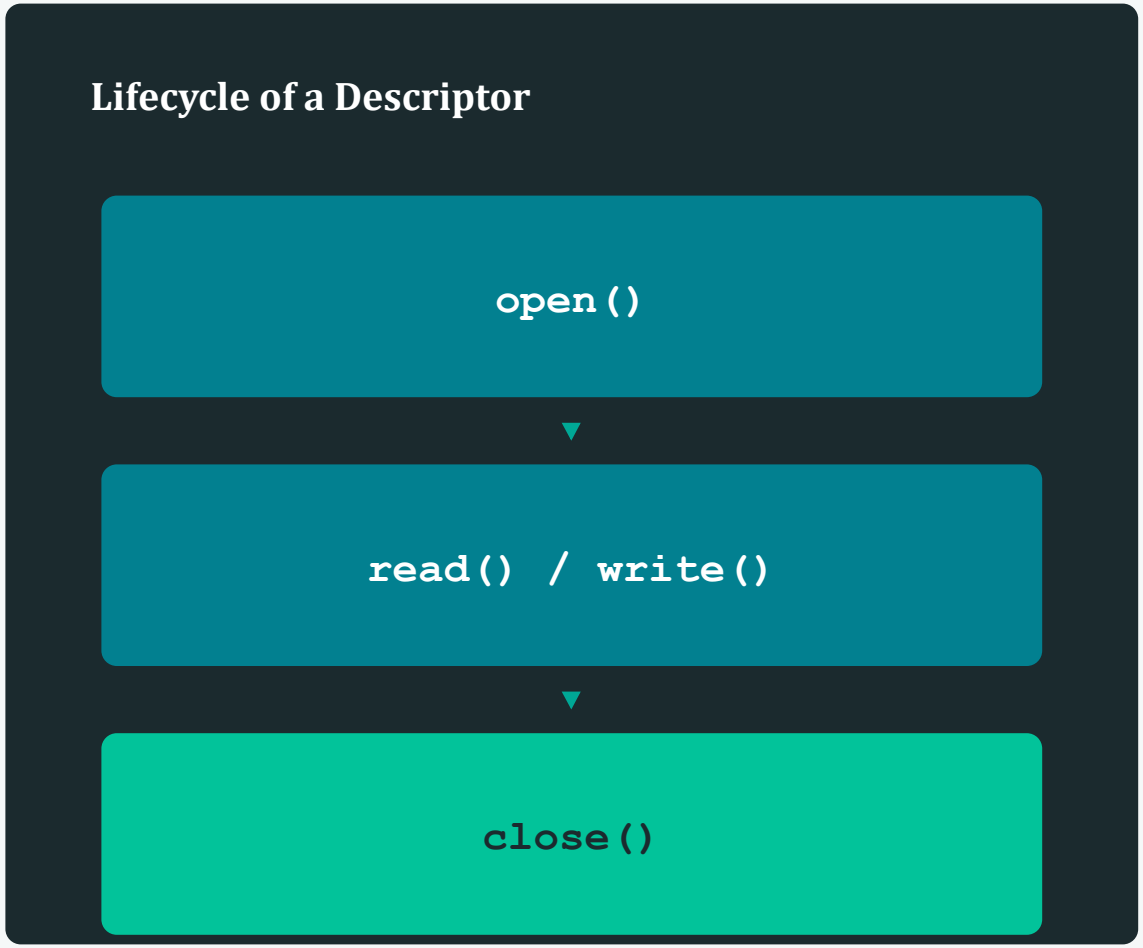
```
write(STDOUT_FILENO, buffer, n);
// writes exactly the n bytes read() just gave us
```

close()

Releases the file descriptor back to the operating system.

```
close(fd);
```

- Every process has a limited table of file descriptors.
- Forgetting close() leaks descriptors — a long-running program can eventually run out and fail to open new files.
- The OS closes everything automatically on exit, but relying on that in a loop or long-lived program is a bug.



Write Then Read a File

Every operation used deliberately, one at a time — file_handling_demo.c

1

open()

Create demo.txt for writing (O_CREAT | O_WRONLY | O_TRUNC)

2

write()

Write the message string into the file

3

close()

Close the write-mode descriptor

4

open()

Reopen demo.txt, this time O_RDONLY

5

read()

Read bytes back into a local buffer

6

close()

Close the read-mode descriptor

Putting It Together: The Read Loop

The same six operations — now streamed in a loop straight to stdout.

```
fd = open(argv[1], O_RDONLY);
if (fd == -1) {
    perror("open");
    return 1;
}

while ((n = read(fd, buffer,
                sizeof(buffer))) > 0)
    write(STDOUT_FILENO, buffer, n);

close(fd);
```

1

One open()

Same fd is reused by every read() and write() call in the loop.

2

The loop condition IS the exit test

read() returns 0 at EOF, which makes the while condition false.

3

No fixed file size needed

Because we loop until read() returns ≤ 0, this works on files of any length.

4

fd → fd

Data moves straight from the input fd to STDOUT_FILENO — no printf needed.

Student Introduction File

Modify the given program so that it creates a file called:

student.txt

and writes the following information into it:

Name: Your Name
Roll No: 101
Course: B.Tech.
Subject: Operating System

Then:

1. Open the file for writing.
2. Write the student information using write().
3. Close the file.
4. Reopen it using open() with O_RDONLY.
5. Read the contents using read().
6. Display the contents on the screen.
7. Close the file.

cat — Simple C Implementation

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int fd, n;
    char buffer[100];

    if (argc != 2) return 1;

    fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    while ((n = read(fd, buffer,
sizeof(buffer))) > 0)
        write(STDOUT_FILENO, buffer, n);

    close(fd);
    return 0;
}
```

touch — write simple C Implementation

Goal: create a file if it does not already exist.

Core function: open()

O_CREAT asks the OS to create the file.

Then close() releases the file descriptor.

wc — Executable C Code

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>

int main(int argc, char *argv[])
{
    int fd, n, in_word = 0;
    char buffer[1024];
    long lines = 0, words = 0, bytes = 0;

    if (argc != 2) return 1;

    fd = open(argv[1], O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }

    while ((n = read(fd, buffer, sizeof(buffer))) > 0)
    {
        bytes += n;

        for (int i = 0; i < n; i++)
        {
            if (buffer[i] == '\n')
                lines++;

            if (isspace((unsigned char)buffer[i]))
                in_word = 0;
            else if (!in_word)
            {
                words++;
                in_word = 1;
            }
        }
    }

    close(fd);

    printf("Lines: %ld\nWords: %ld\nBytes: %ld\n", lines, words, bytes);

    return 0;
}
```

grep : Search for a string inside a file

This introduces string searching and line-by-line processing.

The simple implementation uses C library functions fopen(), fgets(), strstr(), fclose().

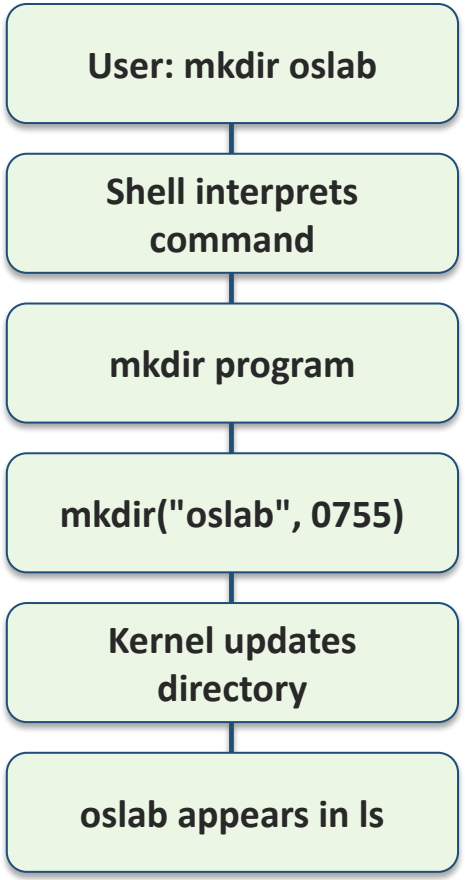
Point out: these are library functions; they ultimately use OS I/O underneath.

MKDIR: Detailed Workflow

mkdir creates a directory entry in the file system.

The path identifies where the directory should be created.

The mode argument supplies initial permission bits, subject to the process umask.



MKDIR: Code with Explanation

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>

int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Usage: ./mymkdir
<directory>\n");
        return 1;
    }

    if (mkdir(argv[1], 0755) == -1) {
        perror("mkdir");
        return 1;
    }

    printf("Created: %s\n", argv[1]);
    return 0;
}
```

argc/argv receive the directory name from the command line.

mkdir(path, mode) asks the OS to create the directory.

Return value -1 indicates an error.

RMDIR : Write code for it

PWD : Write code for it

rm : Write code for it